

Industrial Internship Report on

"Food Delivery Application"

Prepared by

[MD.JAMSHADH]

[23M61A0541]

Executive Summary

This report presents the details of the Industrial Internship offered by Upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). The internship was based on a project/problem statement provided by UCT and was completed.

My project was a Food Delivery Application – Full Stack Java Web App, a web platform connecting customers, restaurants and delivery personnel through one ordering workflow. Restaurant owners can manage food items, customers can browse menus, add items to a cart and place orders, and delivery personnel can accept assigned deliveries and update their progress. Customers can view order status, assigned delivery person and ETA.

The application was developed using Java Servlets, JSP, JSTL and JDBC with an H2 relational database. It uses role-based authentication for Customer, Restaurant Owner and Delivery Person and follows a three-tier structure with JSP/JSTL presentation, Servlet-based processing and DAO/JDBC data access.

This internship gave me practical exposure to requirement analysis, web development, authentication, testing and debugging, while showing me how a complete full-stack Java application can be developed within a fixed project scope.

TABLE OF CONTENTS

1	Preface.....	3-4
2	Introduction.....	4-9
2.1	About UniConverge Technologies Pvt Ltd.....	4-5
2.2	About upskill Campus.....	5-6
2.3	The IoT Academy.....	6-7
2.4	Objective.....	7-8
2.5	Glossary.....	8-9
3	Problem Statement.....	9-10
4	Existing and Proposed solution.....	9-11
5	Proposed Design/ Model.....	11-17
5.1	High Level Diagram (if applicable).....	14-15
5.2	Low Level Diagram (if applicable).....	15-16
5.3	Interfaces (if applicable).....	16-17
6	Performance Test.....	17-21
6.1	Test Plan/ Test Cases.....	19
6.2	Test Procedure.....	19-20
6.3	Performance Outcome.....	20-21
7	My learnings.....	21-29
8	Future work scope.....	29-31

1.Preface

Over the course of this six-week internship, I worked on designing, implementing and testing a complete Food Delivery Application using Java full-stack technologies. The project was treated as an industry-oriented problem with a defined scope, timeline and deliverable.

An internship of this kind is important for career development because it connects classroom concepts with practical software development. In this project, Java programming, web requests, JSP views, database operations, authentication, sessions and testing had to work together as one system rather than as separate academic topics.

The problem statement given by UCT was to design and build a Food Delivery Application that forms a bridge between restaurants and customers. Restaurant owners should be able to register and manage their food menu, customers should be able to browse restaurants and order food, and delivery personnel should be able to fulfil assigned deliveries.

The implementation was intentionally scoped to the fundamental ordering workflow. The project covers role-based registration and login, restaurant and menu management, customer cart and checkout, order creation, restaurant acceptance, delivery-person assignment, ETA information, delivery status updates and customer order tracking.

The program was planned around weekly milestones covering requirement understanding, system design, database implementation, backend and JSP development, testing, debugging and documentation. Dividing the project into these stages helped maintain progress within the six-week period.

Through this internship I learned how to translate a written requirement into a working web application. I gained practical experience with Java Servlets, JSP, JSTL, JDBC, SQL, session management, role-based access control, DAO-based database interaction, Maven and local Tomcat execution.

The project also improved my debugging approach. Runtime issues involving nested servlet paths and stylesheet references demonstrated that an application can compile successfully and

still fail after deployment. Investigating request paths and correcting root-relative or context-path-relative references helped me understand practical web application behaviour.

I would like to thank Upskill Campus, The IoT Academy and UniConverge Technologies Pvt Ltd for providing the internship opportunity and project problem statement, and the mentors and team members who provided guidance during the internship.

To my juniors and peers, my suggestion is to understand the problem statement before writing code, divide the application into manageable modules, keep database and presentation responsibilities separate, test each workflow step, and document important decisions during development rather than only at the end.

Overall, the internship gave me a practical understanding of how a full-stack Java application is planned, developed, tested and documented. The Food Delivery Application strengthened my understanding of web development and gave me greater confidence in approaching larger software projects.

2. Introduction

2.1 About UniConverge Technologies Pvt Ltd



The slide features the UniConverge Technologies logo at the top center, set against a background of a modern industrial factory. Below the logo, three rounded rectangular boxes are arranged horizontally, each with a red header and a black body. The first box, titled 'IIOT Products', lists offerings such as Remote IOs, Wireless IOs, LoRaWAN Sensor Nodes/ Gateways, Signal converter, and IoT gateways. The second box, titled 'IIOT Solutions', lists offerings like OEE, Predictive Maintenance, LoRaWAN based Remote Monitoring, IoT Platform, and Business Intelligence. The third box, titled 'OEM Services', states that they offer solutions ranging from product design to final production.

uct
Uniconverge Technologies

IIOT Products
We offer product ranging from Remote IOs, Wireless IOs, LoRaWAN Sensor Nodes/ Gateways, Signal converter and IoT gateways

IIOT Solutions
We offer solutions like OEE, Predictive Maintenance, LoRaWAN based Remote Monitoring, IoT Platform, Business Intelligence...

OEM Services
We offer solutions ranging from product design to final production we handle everything for you..

UniConverge Technologies Pvt Ltd (UCT) is a company established in 2013 and works in the Digital Transformation domain, providing industrial solutions with a prime focus on sustainability and Return on Investment (RoI).

UCT works with modern technologies and technical areas including Internet of Things, Cyber Security, Cloud Computing platforms such as AWS and Azure, Machine Learning, communication technologies such as 4G, 5G and LoRaWAN, Java Full Stack development, Python and front-end development.

The internship project provided an opportunity to work on a Java Full Stack web-development problem. The Food Delivery Application is relevant to this area because it requires a browser interface, server-side request processing, business rules, authentication, session management and relational data persistence.

The project demonstrated that software development is not limited to writing source code. A complete application requires requirement analysis, architecture design, database modelling, interface development, validation, testing, debugging and documentation. Each activity affects the reliability of the final system.

The problem statement required three practical user roles: Customer, Restaurant Owner and Delivery Person. Each role has different responsibilities, so the application had to provide role-specific dashboards and protect operations that should not be available to other roles.

The application also demonstrated the importance of modular development. Authentication, restaurant management, menu management, cart processing, checkout, order processing and delivery operations were treated as separate responsibilities connected through the overall workflow.

The project also gave practical exposure to the concept of a multi-role application. Although all three roles use the same application, their goals are different. A customer is concerned with discovering food and receiving an order, a restaurant owner is concerned with menu and

order management, and a delivery person is concerned with completing assigned deliveries. Designing the application around these responsibilities helped me understand why role-specific views are useful.

Another important consideration was maintaining a consistent vocabulary throughout the application. Terms such as restaurant, food item, order, order item, delivery person, status and ETA appear in multiple modules. Using consistent names across the database, Java classes, servlet routes and JSP pages reduces confusion and makes the project easier to maintain.

The project also required thinking about the sequence in which data becomes available. A customer cannot create a meaningful order without first selecting a restaurant and food items. A restaurant cannot assign a delivery person until an order exists. A customer should not see an out-for-delivery state until the delivery person has performed the pickup action. Understanding these dependencies helped shape the application flow.

From a development perspective, the internship demonstrated why a small application still benefits from architecture. Without separation between the presentation, controller and database layers, changes to one feature would be more likely to affect unrelated parts of the system. The chosen structure provides a clear path for future improvements.

2.2 About Upskill Campus (USC)

Upskill Campus, along with The IoT Academy and UniConverge Technologies, facilitated the execution of the internship program. Upskill Campus is a career-development platform focused on personalised, scalable and measurable career-development opportunities.

The internship provided a structured environment in which the project could be completed within a defined period. Instead of focusing only on theoretical concepts, the internship involved practical implementation through a project-based approach.

The Food Delivery Application required the application of Java programming, web-development and database-management knowledge to solve a practical problem. The project was divided into requirement understanding, design, implementation, testing and documentation stages.

One benefit of the internship structure was the opportunity to understand how different software-development areas depend on each other. Database design had to match the order workflow, authentication had to match role-specific access, and the JSP pages had to receive correct data from the server.

The internship also developed problem-solving skills. Errors can occur in SQL statements, URL mappings, sessions, JSP rendering or database operations. Identifying the cause, applying a controlled correction and retesting are important professional development practices.

Working within a fixed schedule also taught me to prioritise the core workflow before optional features. This is important in real projects because a smaller stable system is more useful than a larger incomplete feature set.

2.3 The IoT Academy

The IoT Academy is the EdTech division of UCT and conducts long-term executive certification programs in collaboration with EICT Academy, IIT Kanpur, IIT Roorkee and IIT Guwahati across multiple domains.

As part of the internship arrangement, The IoT Academy contributed to the structured learning environment through which the project was carried out. The internship provided an opportunity to work on a defined software problem while gaining practical exposure to development concepts.

The Food Delivery Application provided exposure to Java application development, web technologies, relational databases, authentication, session handling, MVC-style separation, SQL, JDBC and software testing.

The project also demonstrated that knowing programming syntax alone is not sufficient to develop a complete application. The developer must understand HTTP requests, sessions, database relationships, validation, access control and the sequence of business actions.

The internship therefore provided an opportunity to understand application development from the initial problem statement through implementation, testing and documentation. It also highlighted the value of reproducible development using Maven, an embedded Tomcat environment and H2 database.

2.4 Objectives of this Internship program

The primary objective of the internship was to gain practical experience in developing a complete web application based on an industry-oriented problem statement. The Food Delivery Application required functional, database, usability and access-control considerations. The major objectives were:

- To gain practical experience of working on an industry-oriented full-stack Java software project.
- To understand the process of converting a written problem statement into a working web application.
- To improve practical knowledge of Java Servlets, JSP and JSTL.
- To understand the role of JDBC in connecting Java code with a relational database.
- To design and use a relational schema for users, restaurants, food items, orders and delivery personnel.
- To understand and implement CRUD operations for restaurant menu management.
- To implement role-based registration, login and dashboard redirection.

- To use sessions and an authentication filter to protect logged-in-only routes.
- To implement customer restaurant and menu browsing.
- To implement a session-based shopping cart and checkout process.
- To maintain consistency between orders and order_items during checkout.
- To implement restaurant order acceptance, delivery-person assignment and ETA entry.
- To implement delivery-person actions for pickup and delivery.
- To display order status and delivery information to the customer.
- To use PreparedStatements and DAO classes for safer and maintainable database access.
- To understand separation between presentation, controller and data-access responsibilities.
- To perform functional testing using multiple application roles.
- To identify and correct runtime issues involving URL paths, request dispatching and stylesheet loading.
- To improve debugging, problem-solving, communication and documentation skills.
- To understand how a large feature brief can be scoped into an achievable six-week implementation.
- The internship also aimed to improve planning, communication, time management and systematic problem solving. Completing the project within a fixed period required the work to be divided into stages and completed progressively.

[1] Oracle Java Servlet and JSP documentation -

<https://www.oracle.com/java/technologies/servlet-technology.html>

[2] Apache Tomcat documentation -

<https://tomcat.apache.org/tomcat-11.0-doc/>

[3] H2 Database documentation -

<https://www.h2database.com/>

2.5 Glossary

Term	Meaning
JDBC	Java Database Connectivity - API used by Java code to connect to and execute SQL against a relational database.
Servlet	A Java class that handles HTTP requests and responses on the server side.
JSP	Java Server Pages - technology used to create dynamic HTML views.
JSTL	JSP Standard Tag Library - tag-based support used to keep presentation logic cleaner.
DAO	Data Access Object - design pattern separating database access from controller and presentation logic.
CRUD	Create, Read, Update, Delete - basic database operations.
ETA	Estimated Time of Arrival - estimated delivery time shown for an order.
AuthFilter	Servlet filter used to protect routes requiring an authenticated session.
Session	Server-side state associated with a user interaction, used for login state and cart information.
Prepared Statement	JDBC mechanism for parameterised SQL statements.
H2	Lightweight relational database used for local development and testing.
MVC	Model-View-Controller - separation of model, controller and presentation responsibilities.

The application therefore had to solve more than simple food listing. It had to coordinate a sequence of actions performed by different users at different times. This made state management and database relationships important parts of the problem.

Another requirement was that the application remain understandable for a student-level full-stack implementation. The design avoided unnecessary production-scale infrastructure and concentrated on the core entities and operations required to demonstrate the complete use case.

Correctness also includes protecting access to role-specific functionality. A customer should not be able to use a restaurant menu-management route simply by knowing its URL, and an unauthenticated visitor should not be able to access the cart or order history. These requirements influenced the authentication and filtering design.

3.Problem Statement

The assigned problem statement was to design and build a Food Delivery Application that forms a bridge between restaurants and consumers. The application needed to represent three practical roles: Customer, Restaurant Owner and Delivery Person.

Restaurant owners should be able to register, maintain restaurant information and manage food items with fields such as name, description, price, availability and image information. Customers should be able to register, view restaurants, browse menus, add food to a cart and place an order.

After a customer places an order, the restaurant should be able to accept it. During acceptance, an available delivery person can be assigned and an estimated delivery time can be recorded. The delivery person should then be able to view the assigned order and update its progress through pickup and delivery.

The customer-facing order history should reflect the order lifecycle. The implemented flow is PLACED, CONFIRMED, OUT_FOR_DELIVERY and DELIVERED, with cancellation also possible from the placed state. The customer can see the assigned delivery person's identity and ETA after confirmation.

The cart and checkout flow must remain consistent. Adding the same food item repeatedly should update its quantity, and checkout must create an order and related order-item records correctly. The cart is cleared after successful checkout so that the same items are not accidentally submitted again.

The original problem statement also listed stretch features such as personalised recommendations, ratings and reviews, saved addresses and payment methods, favourites and

recent orders, GPS tracking, notifications, payments and analytics. These were treated as future scope because the internship focused on the core workflow within six weeks.

4.Existing and Proposed solution

Established food-delivery platforms such as Swiggy, Zomato and Uber Eats already provide large-scale solutions for restaurant discovery, ordering, delivery management and customer tracking. Their production systems can include mapping integrations, payment gateways, notifications, recommendations and large-scale infrastructure.

From an internship learning perspective, reproducing a complete production-scale platform is outside the scope of a six-week individual project. The core transferable value therefore lies in correctly implementing the fundamental data model and workflow rather than reproducing production infrastructure.

The proposed solution is a scoped-down but functionally complete full-stack Java web application. It provides separate registration, login and dashboards for Customer, Restaurant Owner and Delivery Person roles.

- Restaurant owners can add, edit, toggle availability and remove menu items. Customers can browse restaurants, view menus, add items to a cart and check out. Restaurants can accept orders, assign an available delivery person and set an ETA. Delivery personnel can update pickup and delivery status.
- The application uses plain JDBC with PreparedStatements and DAO classes rather than an ORM. This is a deliberate value addition because it demonstrates how Java application code communicates with relational tables directly and provides groundwork for later learning of frameworks such as Spring Boot and JPA/Hibernate.
- Role-based authentication and the AuthFilter provide access protection for logged-in routes. The design also keeps the cart, order and order-item workflow consistent and uses an explicit order-status lifecycle so that the system remains understandable.

Code submission (Github link)

<https://github.com/jamshadh19-999/upskillcampus/tree/main/Food-Delivery-App>

Report submission (Githhub Link)

https://github.com/jamshadh19-999/upskillcampus/blob/main/Food-Delivery-App/FoodDeliveryApp_MD_Jamshadh_UCT.pdf.pdf

5.Proposed Design/ Model

The proposed Food Delivery Application is designed as a modular full-stack Java web application connecting customers, restaurants and delivery personnel through a common ordering workflow. It follows a three-tier style consisting of a JSP/JSTL presentation layer, Servlet-based controller/business logic and a JDBC-based data-access layer backed by H2.

The presentation layer contains JSP and JSTL pages for login, registration, restaurants, menus, cart, checkout, orders and dashboards. The pages display prepared data and do not directly perform database operations.

The Servlet layer receives HTTP requests, validates required values, manages sessions, invokes DAO methods and selects the appropriate JSP response. Examples include registration, login, logout, restaurant and menu operations, cart actions, checkout, orders and role-specific dashboards.

The DAO layer isolates SQL operations. UserDAO, RestaurantDAO, FoodItemDAO, OrderDAO and DeliveryPersonDAO represent the main data-access responsibilities. JDBC PreparedStatements are used for parameterised operations.

The data model contains users, restaurants, food_items, delivery_persons, orders and order_items. Foreign-key relationships connect owners to restaurants, restaurants to food items, customers and restaurants to orders, and orders to their individual order items.

The users table stores common account information such as name, email, password, role, phone and address. The restaurants table stores owner information and restaurant details. Food items store the restaurant relationship, name, description, price, availability and image URL.

The orders table stores customer, restaurant and delivery-person relationships together with status, total amount, delivery address, ETA and creation time. The order_items table stores the selected food item, captured name and price, quantity and order relationship.

Authentication establishes a user session and the stored role determines the dashboard. The AuthFilter protects logged-in-only routes such as cart, checkout, orders and restaurant or delivery pages. Unauthenticated requests are redirected to login.

The customer cart is maintained using session state. Adding an item updates its quantity when necessary, while checkout converts the cart into persistent order and order-item records. The cart is cleared after successful order creation.

The restaurant workflow covers menu administration and incoming orders. The restaurant can accept an order, assign an available delivery person and set an ETA. The delivery workflow then moves the order through pickup and final delivery.

The order state flow is PLACED to CONFIRMED, then OUT_FOR_DELIVERY and finally DELIVERED. A placed order may also move to CANCELLED. Keeping these transitions explicit reduces inconsistent status values.

The application includes endpoints such as /register, /login, /logout, /restaurants, /menu, /cart, /checkout, /orders, /restaurant/dashboard, /restaurant/menu, /restaurant/orders/action and /delivery/dashboard. These endpoints provide the main application workflow without exposing database implementation details.

The three-tier arrangement also makes testing more focused. DAO methods can be examined for database behaviour, Servlets can be checked for request and session handling, and JSP pages can be checked for correct display. Testing each layer separately does not replace end-to-end testing, but it helps isolate problems.

The application uses role information as part of the session context so that later requests can be associated with the authenticated user. This is important for operations such as viewing a customer's orders or accessing a restaurant's own menu. The server should use the authenticated context rather than trusting arbitrary identifiers supplied by the browser.

The menu model also separates restaurant ownership from individual food items. A food item references a restaurant, which allows the application to determine which restaurant owns the item when menu operations are performed. This supports the requirement that restaurant owners manage their own menu data.

The order model records both the customer and restaurant. This relationship allows the application to retrieve orders from either side of the transaction. The delivery-person relationship is optional until assignment, which matches the real workflow in which an order can exist before a delivery person is selected.

The order-items model preserves the selected quantity and price for each line. The total amount is associated with the order, while the individual rows provide the detail needed to display what was purchased. This structure also makes the order history easier to reconstruct.

Using H2 for the internship environment keeps setup simple while still providing relational database behaviour. The application code accesses it through JDBC, so the experience remains relevant to other relational databases that may be used during future deployment.

PreparedStatement also make the database code clearer because the SQL statement and its values are handled separately. This is particularly useful for user-entered values such as names, addresses and search text.

5.1 High Level Diagram

The high-level design represents the overall flow from user interaction to persistent data. A browser sends a request to a Servlet, the Servlet performs validation and invokes the relevant DAO, the DAO uses JDBC to communicate with the database, and the result is returned to a JSP view.

For a customer, the flow begins with registration or login, followed by restaurant browsing, menu viewing, cart management and checkout. Checkout creates an order and related order items, after which the customer can view the order history and status.

For a restaurant owner, the flow begins with authentication and the restaurant dashboard. The owner can manage food items and review incoming orders. When an order is accepted, a delivery person can be assigned and an ETA recorded.

For a delivery person, the dashboard displays assigned orders and provides actions for pickup and delivery. After the order is delivered, the delivery person's availability is restored.

The major components are the browser/JSP presentation layer, Servlet controllers, DAO classes, JDBC connection handling and relational database. Authentication and sessions provide access control around protected operations.

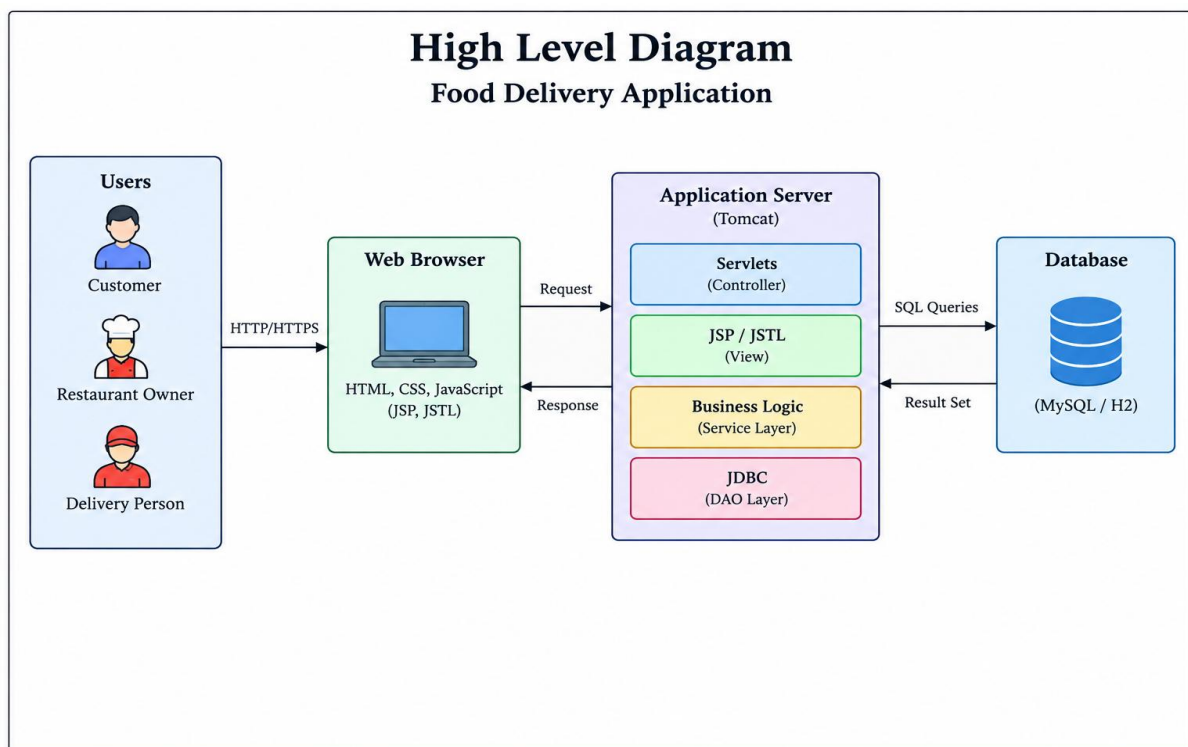
The high-level project diagram has been removed from this template so that the project-specific diagram can be inserted here later.

A customer request for a restaurant menu illustrates the same path at a smaller level. The browser requests the restaurant or menu resource, the Servlet identifies the relevant restaurant, the DAO retrieves matching food items, and the JSP renders those items. This sequence is repeated with different data for other pages.

During checkout, the high-level flow becomes more involved because temporary session data must become persistent database data. The selected food items are read from the cart, the

order total and delivery information are prepared, the order is created, and each selected item is stored as an order-item record.

The restaurant confirmation action demonstrates how the same order record can be updated by another role. The restaurant provides the assignment and ETA information, after which the customer can see the updated state. The delivery person then continues the same workflow.



5.2 Low Level Diagram

The low-level design describes the internal interaction between individual modules. Authentication, menu management, cart processing, order processing and database operations are separated so that each component has a clear responsibility.

The login flow begins with the user submitting credentials to the login Servlet. After successful validation, a session is established and the user's role determines the dashboard. The AuthFilter checks protected routes before the request reaches the relevant page.

The database module communicates through DAO classes. SQL operations insert, retrieve, update and delete records for users, restaurants, food items, orders and delivery personnel. PreparedStatements keep input values separate from SQL syntax.

The cart module maintains selected food items in the customer session. Quantities and totals are updated during shopping, and checkout converts the temporary cart into persistent order and order-item records.

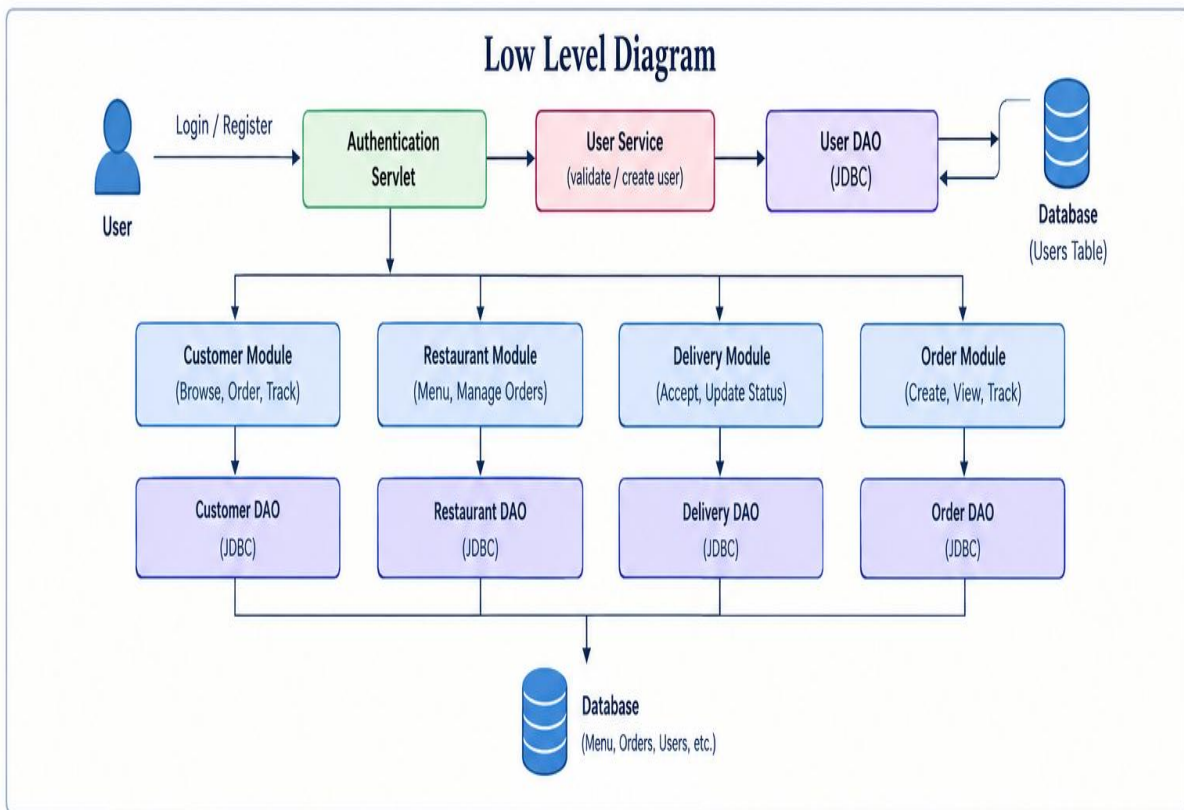
The order module coordinates actions by the restaurant and delivery person. Restaurant acceptance records the assigned delivery person and ETA, pickup changes the state to `OUT_FOR_DELIVERY`, and delivery changes it to `DELIVERED`.

The low-level project diagram has been removed from this template so that the project-specific diagram can be inserted here later.

At a lower level, DAO methods provide the boundary between SQL statements and the rest of the application. This makes it possible to change a query without rewriting the JSP page that displays the result.

The session layer is particularly important for the cart because the cart is temporary user state. The application needs to identify the current user session, retrieve the selected items and update quantities without unnecessarily writing incomplete shopping data into the permanent order tables.

Validation also belongs to the low-level workflow. Required values should be checked before an insert or update is attempted. Numeric values such as price and quantity should be handled consistently, and identifiers should be validated before they are used in database operations. Order-state validation is another low-level concern. The server should determine whether an action is appropriate for the current state instead of assuming that a request received from a browser is valid. This prevents a delivery action from being treated as valid when an order has not reached the appropriate stage.



5.3 Interfaces

The Food Delivery Application provides web interfaces implemented with JSP and JSTL. The interfaces are organised according to the user's role and communicate with Servlet controllers rather than directly accessing the database.

The Login and Registration interfaces allow users to create accounts and authenticate. After login, the stored role determines the dashboard to which the user is directed.

The Customer interface provides restaurant listings and menus. Customers can select a restaurant, inspect available food items and add items to the cart. The cart displays selected items, quantities and the calculated total.

The Checkout interface confirms the information needed to create an order, including the delivery address available to the application. After successful checkout, the order becomes visible in the customer's order history.

The Restaurant Dashboard provides menu management and incoming-order actions. Restaurant owners can add, edit, disable and delete food items and can accept orders, assign delivery personnel and set ETA.

The Delivery Dashboard displays assigned orders and provides pickup and delivery actions. The status changes are reflected in the customer's order view.

The Customer Orders interface displays order history and current status. After confirmation, it also displays the assigned delivery person's identity and ETA.

Role-specific interfaces reduce unnecessary controls. A customer does not need restaurant-management operations, and a delivery person does not need menu-editing operations. This makes the application easier to understand and reduces accidental misuse.

The interfaces were designed to keep the most common user actions visible and understandable. A customer should be able to move from restaurant selection to menu selection, cart review and checkout without needing to understand how the server or database works.

The restaurant interface focuses on management tasks. Menu editing requires clear fields for the food name, description and price, while availability provides a way to control whether an item can currently be ordered. Incoming-order actions are presented separately from menu management because they belong to a different stage of the workflow.

The delivery interface is intentionally smaller in scope. The delivery person mainly needs to know which orders are assigned and which status action is currently valid. This keeps the dashboard focused and reduces unnecessary navigation.

The customer order page is the point where information from all three roles comes together. The order was created by the customer, confirmed by the restaurant, and progressed by the delivery person. Displaying the status and ETA therefore gives the customer visibility into actions performed elsewhere in the system.

6. Performance Test

Performance testing was carried out to verify functional correctness, reliability and basic responsiveness. Because this is a functional single-instance application rather than a production-scale platform, testing concentrated on workflow correctness and local responsiveness.

The main constraints considered were order-state consistency, cart-to-order consistency, correct persistence of order items, restaurant-to-delivery assignment, role-based access and correct rendering of pages under nested URLs.

Authentication tests verified registration and login for all three roles and confirmed correct dashboard redirection. Protected pages were also checked while logged out to confirm AuthFilter redirection.

Menu and cart tests verified adding, editing, disabling and deleting food items, browsing menus, adding multiple items and repeatedly adding the same item. Totals and quantities were checked before checkout.

Checkout and order tests verified that order and order-item records were saved correctly and that the cart was cleared after successful checkout. Restaurant tests verified acceptance, delivery-person assignment and ETA entry.

Delivery tests verified pickup and final delivery. The customer's order page reflected the updated state and the delivery person's availability changed back to Available after completion.

Testing was performed locally through an embedded Tomcat instance using the tomcat7-maven-plugin and an embedded H2 file database. Large-scale load testing and cloud deployment benchmarking were outside the project scope.

The most important constraints for this application were correct order-state transitions, consistency between the cart and order records, protected access and reliable rendering of nested routes. These were addressed through separated DAO operations, sessions, AuthFilter and repeated end-to-end testing.

Testing also considered the consistency of information displayed at different stages. The menu price used during shopping should contribute correctly to the cart total, and the order created during checkout should contain the selected quantities and prices. These checks help identify problems that might not be visible from a single page.

Another important test area was role isolation. A successful login as one role should not simply expose all dashboards. The application was therefore checked using separate role sessions to verify that the expected dashboard and protected routes were associated with the authenticated account.

The local test environment was useful because it allowed repeated execution of the same scenarios without depending on external services. This made it possible to reproduce bugs, make a change and execute the affected scenario again.

Performance in this project was treated as a practical responsiveness requirement rather than a numerical benchmark. For a local single-user application, pages should respond promptly enough for manual testing, while correctness and data consistency remain more important than attempting to optimise a workload that was not part of the internship scope.

The absence of large-scale load testing is itself a limitation that should be documented. A production system would need additional testing for concurrent customers, simultaneous restaurant actions, delivery assignment races, database connection limits and network latency. The testing process also showed the value of realistic test data. A workflow involving one restaurant and one food item may appear correct even when relationships between multiple

restaurants, multiple items and multiple users contain errors. Testing with more than one role and more than one item provides a stronger functional check.

6.1 Test Plan/ Test Cases

Register as a Customer, Restaurant Owner and Delivery Person and confirm correct dashboard redirection after login.

- Attempt to access a protected page such as /cart while logged out and confirm redirection to the login page.
- Restaurant owner adds a new food item and confirm that it appears on the customer-facing menu.
- Restaurant owner edits a food item and confirm that the updated information is reflected.
- Restaurant owner disables a food item and confirm the availability state is reflected correctly.
- Restaurant owner deletes a food item and confirm that it is removed from the menu.
- Customer browses restaurants and opens a selected restaurant menu.
- Customer adds multiple different food items and verifies quantities and total amount.
- Customer adds the same food item repeatedly and verifies quantity handling.
- Customer completes checkout and confirms that an order and order-item records are created.

Customer verifies that the cart is cleared after successful checkout.

Restaurant accepts an incoming order, assigns an available delivery person and sets an ETA. Confirm that the assigned delivery person becomes busy and is not offered for another assignment.

Delivery person views the assigned order and marks it as picked up.

Confirm that pickup changes the order to OUT_FOR_DELIVERY.

Delivery person marks the order as delivered and confirm the final status.

Confirm that the delivery person's availability returns to Available after delivery.

Customer opens My Orders and verifies assigned delivery person and ETA after confirmation.

Attempt an unexpected order-state transition and verify that the application does not silently create an inconsistent state.

Verify that nested restaurant and delivery URLs load the correct JSP pages and stylesheet resources.

6.2 Test Procedure

The test cases were executed manually using multiple browser sessions, with separate sessions representing the customer, restaurant owner and delivery person. This made it possible to observe how actions performed by one role appeared in another role's interface.

The application was run locally through the Maven-managed embedded Tomcat environment. The H2 file database provided persistence between requests so restaurant, menu, customer, delivery and order operations could be tested across the complete workflow.

For each scenario, the expected result was identified before the action was performed. The actual result was compared with the expected state in the interface and stored application data. When an issue was found, the request path, servlet mapping, DAO operation or JSP output was examined before applying a correction.

6.3 Performance Outcome

The performance outcome was evaluated mainly through functional correctness and local responsiveness rather than large-scale load measurements. The complete workflow was executed from registration and login through restaurant browsing, cart creation, checkout, restaurant acceptance, delivery assignment, pickup and final delivery.

The tests confirmed role separation, menu operations, cart calculations, checkout persistence and order lifecycle behaviour. Order and order-item records were checked to ensure that selected items and quantities were retained after checkout.

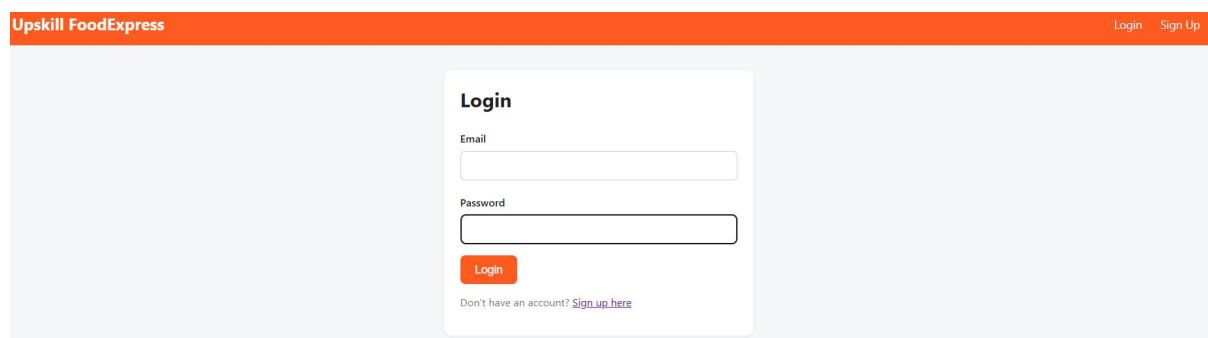
The delivery workflow was tested by moving an assigned order through pickup and delivery. The customer's order page reflected the updated state, while the delivery person's availability changed back to Available after completion.

Two runtime issues were discovered and corrected during testing. Three servlets initially used relative RequestDispatcher paths under nested URLs such as /restaurant/dashboard, producing HTTP 404 responses. The paths were changed to root-relative dispatcher paths.

A related stylesheet path issue was corrected using a context-path-relative stylesheet reference so that CSS loaded correctly under nested routes.

After these corrections, the affected dashboard pages loaded correctly and order-status updates on the local single-user environment were effectively instantaneous. This result is appropriate for a local test environment and should not be interpreted as a production-scale benchmark.

Testing demonstrated the importance of checking complete workflows instead of pages individually. A cart can appear correct while checkout fails, or a restaurant can update an order while the customer page fails to retrieve the new status. End-to-end testing therefore provided stronger evidence of correctness.



Create an account

I am a

Customer (order food) ✓

Customer (order food)

Restaurant Owner

Delivery Person

Email

Password

Phone

Address

Create Account

Already have an account? [Login](#)

7. Learnings

I learned that a full-stack application is essentially a chain of responsibilities. A change in one layer can affect the next layer, so understanding the interfaces between layers is as important as understanding the individual components.

The project improved my understanding of HTTP request-response behaviour. Each button or form submission ultimately becomes a request to a server-side endpoint. The Servlet mapping determines which Java class handles that request, and the resulting response determines which page the user sees next.

I became more comfortable reading stack traces and server errors. A 404 response is different from a database error, and a JSP rendering problem is different from a failed authentication check. Classifying the error before attempting a solution made debugging more systematic.

Testing improved my confidence because it provided evidence that the application worked across several role combinations. Repeating the same scenario after a code change also reduced the risk of fixing one feature while accidentally breaking another.

i The delivery workflow starts only after assignment. Understanding these separate sequences helped me avoid treating all users as if they interacted with the application in the same way.

I also learned that ETA is different from an exact delivery time. The project stores an estimated delivery duration so that the customer receives useful information while the actual completion time remains dependent on the delivery process.

The project gave me experience in handling availability as a state rather than simply as a text field. Food items can be available or unavailable, and delivery personnel can be available or busy. These states influence what actions the system should offer.

I learned that a state value should have a limited and understandable set of possible values. The order lifecycle is easier to reason about when the application expects known states instead of allowing arbitrary strings. This reduces ambiguity in both code and database records.

The experience also made me more aware of the difference between development convenience and production requirements. H2 and embedded Tomcat are convenient for an

internship project, but a production platform would require stronger deployment, monitoring, backup, security and scalability arrangements.

Documentation helped me understand the application from a different perspective. While coding, I often think about the immediate next step. While writing the report, I had to think about the complete system and explain how the parts connect. This improved my ability to communicate technical work.

I also learned that a project report should record limitations honestly. The application does not claim to provide live GPS tracking, online payments or large-scale load handling because those capabilities were outside the implemented scope. Identifying limitations makes the project description more accurate.

- One of the most important learnings was understanding how a Java web request travels through an application. A browser request reaches a Servlet, the Servlet performs validation and invokes the appropriate data-access operation, the DAO communicates with the database through JDBC, and the result is then returned to a JSP view. Seeing this flow in a working application made the relationship between different layers much clearer.
- I learned the practical value of separating presentation, controller and data-access responsibilities. JSP pages should concentrate on displaying information, Servlets should coordinate requests and application actions, and DAO classes should contain the SQL interaction. This separation makes the code easier to understand because each component has a defined purpose.
- Working with JDBC gave me a better understanding of how Java applications interact with relational databases. Instead of relying on an ORM to hide SQL operations, this project used PreparedStatement and DAO classes. This helped me understand connections, queries, parameters, result sets and the mapping between database records and Java objects.
- Database design was another important learning outcome. The application required separate entities for users, restaurants, food items, delivery persons, orders and order

items. I learned that the structure of these tables must reflect the actual business relationships. For example, one restaurant can have many food items, and one order can contain many order items.

- I also learned why an `order_items` table is necessary instead of storing all selected food information directly inside an order record. An order represents the overall transaction, while order items represent the individual products, quantities and captured prices. This separation makes it possible to display detailed order contents and maintain a clear relational structure.

Implementing the customer cart taught me about temporary session state. The cart exists before an order is created, so it cannot be treated in exactly the same way as permanent database records. The application needs to maintain quantities and totals during the user's interaction and then convert that information into persistent order records at checkout.

8. Future work scope

A future version could support a dedicated restaurant onboarding process in which restaurant details are validated before the restaurant becomes visible to customers.

Restaurant owners could receive a more detailed order queue showing newly placed, confirmed, prepared and completed stages if additional preparation states are introduced.

Food items could include category information so that customers can filter menus by type such as starters, main course, beverages or desserts.

Search and filtering could be added to restaurant and menu pages to help customers locate relevant options more quickly as the number of restaurants grows.

Customers could receive clearer checkout validation for missing or invalid delivery information before an order is created.

The application could support order cancellation rules that depend on the current order state and the restaurant's processing stage.

Delivery personnel could have an availability toggle so that the system does not assign new orders when the person is temporarily unavailable.

- Live GPS-based delivery tracking could be integrated using a mapping service such as Google Maps or Mapbox.
- Route optimisation could be added to improve delivery planning when multiple orders or delivery points are considered.
- An online payment gateway could be integrated so customers can use supported digital payment methods in addition to pay-on-delivery.
- Ratings and reviews could be added for restaurants and dishes after order completion.
- A personalised recommendation engine could suggest dishes or restaurants using customer order history.
- Saved delivery addresses could allow customers to select frequently used locations without entering them repeatedly.
- Saved payment methods could be supported through a secure payment provider with appropriate controls.
- Favourite restaurants, favourite dishes and recent-order shortcuts could make repeat ordering more convenient.
- Push notifications could inform customers about order placement, confirmation, pickup, delivery and completion.
- An administration and analytics panel could provide platform-wide information about dishes, customers, orders and restaurant activity.
- Richer dish and restaurant media could be supported with validated photographs and additional menu information.
- Concurrency and transaction handling could be strengthened for simultaneous checkout, restaurant actions and delivery assignment.
- The H2 development database could be migrated to a production relational database such as MySQL or PostgreSQL.

- The raw JDBC DAO layer could later be migrated to an ORM such as Hibernate/JPA for comparison with framework-based persistence.
- The backend could be migrated to Spring Boot while preserving the same domain entities and business workflows.
- REST APIs could be introduced so the backend can also support mobile or modern frontend clients.
- Role and permission management could be made more granular for future administrative responsibilities.
- Production security could be strengthened through secure session configuration, input validation and deployment controls.
- Automated integration and end-to-end tests could be added so the complete order workflow can be executed repeatedly after changes.